Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: "Informații invizibile: forensics și steganografie"

A efectuat : st. gr. SI–262
Inculeț Andrian

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Pregătire (comună celor două lucrări)

Intră în depozitul disciplinei, activează mediul virtual și instalează biblioteca de imagini. Apoi rulează scriptul de pregătire de mai jos: el îți creează, chiar la tine, fișierele pe care le vei analiza (un șir codat în straturi, un mesaj XOR, o poză cu EXIF și GPS, și o poză cu un fișier ascuns). Astfel toată lumea lucrează pe aceleași ținte, repetabil.

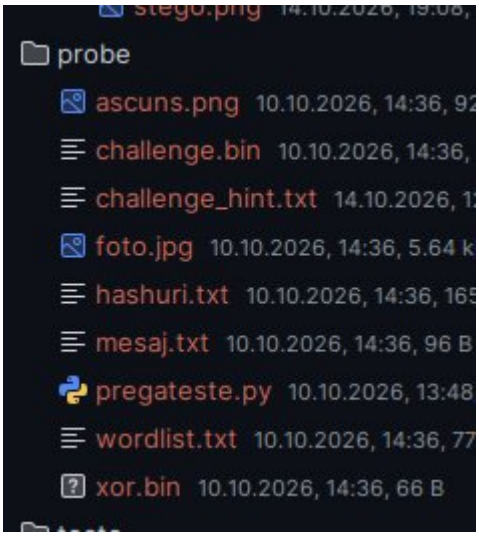
```
cd tehnici-prog-<nume>
# activeaza mediul (Windows PowerShell):
.\.venv\Scripts\Activate.ps1
# sau Linux:  source .venv/bin/activate

pip install pillow
mkdir -p arsenal/forensics teste probe

# creeaza fisierele de laborator:
python probe/pregateste.py     # scrie in probe/
```

Scriptul probe/pregateste.py îți este dat (în depozitul de start al laboratorului). El produce: probe/mesaj.txt (șir în mai multe straturi de codare), probe/xor.bin (text criptat cu XOR pe un octet), probe/hashuri.txt (câteva hash-uri de spart), probe/foto.jpg (cu EXIF și GPS) și probe/ascuns.png (cu o arhivă lipită la coadă). Dacă lipsește, cere-l asistentului.

Ce poate crăpa. Dacă import PIL dă eroare, biblioteca se numește la instalare pillow (nu „PIL"). Dacă nu vezi (.venv) în prompt, mediul nu e activat.



LUCRAREA A (nucleu). Decoderul universal: arsenal/forensics/decode.py

Obiectiv: un program care primește un șir sau un fișier, recunoaște stratul și îl desface; iar pentru un XOR cu cheie de un octet sau un hash de parolă, îl sparge singur. Pornești de la scheletul de mai jos; funcțiile marcate cu TODO sunt ale tale.

A1. Text și octeți: încălzirea (10 min)

Creează arsenal/forensics/decode.py. Fixează-ți întâi diferența dintre text și octeți, de care atârnă tot.

```
s = "Salut"
b = s.encode()        # bytes
print(b, b[0])        # b"Salut" 83
# TODO: tipareste b.hex() si refa bytes din hex
#       cu bytes.fromhex(...)
```

De observat. b[0] e un număr, nu o literă. Tot ce urmează lucrează pe bytes, nu pe text.

A2. Recunoști stratul după formă (14 min)

Scrie o funcție care, dat un șir, ghicește ce codare e, uitându-se la caractere. Nu decoda încă, doar clasifică.

```
import re
def ghici_strat(s):
    # TODO: daca are %xx -> "url"
    # TODO: daca doar 0-9 a-f si lungime para -> "hex"
    # TODO: daca doar A-Za-z0-9+/= -> "base64"
    return "necunoscut"
print(ghici_strat("%2Fetc"))   # url
print(ghici_strat("48656c6c6f"))  # hex
```

De observat. Ordinea contează: verifică întâi „%" (URL), apoi hexul (mai strict), apoi base64 (cel mai larg alfabet).

A3. Desfaci stratul recunoscut (14 min)

Acum chiar decodează, în funcție de ce a zis A2. Întoarce și tipul, și rezultatul.

```
import base64
from urllib.parse import unquote
def desfa(s):
    strat = ghici_strat(s)
    # TODO: pentru fiecare strat, decodeaza
    #   base64 -> base64.b64decode(s)
    #   hex    -> bytes.fromhex(s)
    #   url    -> unquote(s).encode()
    # intoarce (strat, rezultat_bytes)
    return strat, s.encode()
```

A4. Straturi multiple, într-o buclă (14 min)

Un rezultat poate fi tot o codare. Decodează în buclă până iese text citibil (sau te oprești după câteva încercări, ca să nu intri în buclă infinită).

```
def citibil(b):
    # TODO: True daca toti octetii sunt 32..126
    return all(32 <= c < 127 for c in b)

val = "NTM1OTU3NTk="
```

```
for _ in range(8):
    strat, b = desfa(val)
    # TODO: daca citibil(b): opreste si tipareste
    #       altfel val = b.decode(); continua
```

A5. Spargi un XOR cu cheie de un octet (16 min)

Citește probe/xor.bin (octeți), încearcă toate cele 256 de chei și păstrează rezultatul care arată a text.

```
date = open("probe/xor.bin", "rb").read()
for k in range(256):
    clar = bytes(b ^ k for b in date)
    # TODO: daca citibil(clar):
    #       print("cheie:", k, clar.decode())
```

De observat. Operatorul ^ e XOR; b ^ k aplică cheia pe un octet. O cheie de un octet are fix 256 de valori, deci le încerci pe toate.

A6. Spargi un hash prin dicționar (16 min)

Din probe/hashuri.txt ia un hash. Identifică algoritmul după lungime, apoi caută parola într-o wordlist.

```
import hashlib
tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256
for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
    # TODO: daca hashlib.md5(cuv.encode()).hexdigest()
    #       == tinta: print("gasit:", cuv); break
```

De observat. Nu „decriptezi" hash-ul: hashuiești fiecare cuvânt și compari. Lungimea îți spune ce algoritm să folosești.

A7. argparse, fișa JSON și un test (16 min)

Leagă totul într-o unealtă comandată din afară, salvează rezultatul ca fișă, și scrie un test.

```
import argparse, json
p = argparse.ArgumentParser()
p.add_argument("intrare")        # sir sau cale
p.add_argument("--fisier", action="store_true")
a = p.parse_args()
# TODO: daca a.fisier, citeste octetii din cale;
#       altfel trateaza intrarea ca sir
# TODO: salveaza {strat, rezultat} in
#       arsenal/forensics/fisa.json
```

Un test scurt care ține decoderul cinstit:

```
# teste/test_decode.py
from arsenal.forensics.decode import desfa
def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

Predai: decode.py, fisa.json, teste/test_decode.py, plus în raport.md: ce ascundea mesajul cu straturi, ce cheie XOR ai găsit și ce parolă ai spart.

decode.py

```
import argparse
import base64
import hashlib
import json
import os
import re
from urllib.parse import unquote

print("A1:")
s = "Salut"
b = s.encode()          # bytes
print(b, b[0])          # b"Salut" 83

h = b.hex()
print(bytes.fromhex(h))

print("\nA2:")
def ghici_strat(s: str) -> str:
    if re.fullmatch("%[a-fA-F0-9]{2}", s):
        return "url"
    elif re.fullmatch("[0-9a-fA-F]+",s) and len(s) % 2 == 0:
        return "hex"
    elif re.fullmatch("[A-Za-z0-9+/]+={0,2}",s):
        return "base64"
    else:
        return "necunoscut"

print("%2Fetc: ", ghici_strat("%2F"))   # url
print("48656c6c6f: ",ghici_strat("48656c6c6f"))  # hex

print("\nA3:")
def desfa(s: str) -> tuple[str, bytes | None]:
    strat = ghici_strat(s)

    match strat:
        case "url":
            return strat, unquote(s).encode()
        case "hex":
            return strat, bytes.fromhex(s)
        case "base64":
            return strat, base64.b64decode(s)
    return strat, None

print(desfa("48656c6c6f"))


def citibil(b: bytes) -> bool:
    # TODO: True daca toti octetii sunt 32..126
    return all(32 <= c < 127 for c in b)


if __name__ == "__main__":
    print("\nA4:")

    val = "NTM1OTU3NTk="
    for _ in range(8):
        strat, b = desfa(val)
        if b is None:
```

```
            break
        nou = b.decode("latin-1")
        if "necunoscut" == ghici_strat(nou):
            print(nou)
            break
        else:
            val = nou

    print("\nA5:")

    date = open("../../probe/xor.bin", "rb").read()
    for k in range(256):
        clar = bytes(b ^ k for b in date)
        if citibil(clar):
            print("cheie: ", k , ", ", clar.decode())
            break

    print("\nA6:")
    tinta = open("../../probe/hashuri.txt").readline().strip()
    print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256
    for cuv in open("../../probe/wordlist.txt", encoding="latin1"):
        cuv = cuv.strip()

        if hashlib.md5(cuv.encode()).hexdigest() == tinta:
            print("Gasit: ", cuv)
            break

    print("\nA7:")

    p = argparse.ArgumentParser()
    p.add_argument("intrare")
    p.add_argument("--fisier", action="store_true")
    a = p.parse_args()

    if a.fisier:
        with open(a.intrare, "rb") as f:
            val = f.read()
    else:
        val = a.intrare

    val = val.decode()

    fisa_strat = "necunoscut"
    for _ in range(8):
        strat, b = desfa(val)
        if b is None:
            break
        val = b.decode("latin-1")
        fisa_strat = strat
        if citibil(b) and ghici_strat(val) == "necunoscut":
            break




    if not citibil(val.encode("latin-1")):
        print("Rezultatul final nu este complet citibil ca text simplu.")
    else:
        print("Rezultat final decodat:", val)
```

```
        fisa = {
            "strat": fisa_strat,
            "rezultat": val
        }

        os.makedirs("../../arsenal/forensics", exist_ok=True)
        with open("../../arsenal/forensics/fisa.json", "w", encoding="utf-8") as f:
            json.dump(fisa, f, indent=4)

        print("Rezultat salvat cu succes în fisa.json")


    print("\nextra punct:")
    with open("../../probe/challenge.bin", "rb") as f:
        secret = f.read().decode()
        b64_bytes = base64.b64decode(secret)
        decoded = ""
        for byte in b64_bytes:
            decoded += chr(byte - 128)

        print(decoded)
```

Output:

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-
programare/Laborator6/arsenal/forensics$ python decode.py ../../probe/mesaj.txt --
fisier
A1:
b'Salut' 83
b'Salut'

A2:
%2Fetc:  url
48656c6c6f:  hex

A3:
('hex', b'Hello')

A4:
SYWY

A5:
cheie:  128 ,  Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}

A6:
32
Gasit:  password

A7:
Rezultat final decodat: FLAG{straturi_de_encoding}
Rezultat salvat cu succes în fisa.json

extra punct:
FLAG{lant_complet_despicat}
```

fisa.json

```
{
    "strat": "hex",
    "rezultat": "FLAG{straturi_de_encoding}"
}
```

raport.md

```
### 1. Ce ascunde mesajul, strat cu strat

Mesajul din probe/mesaj.txt e codat in mai multe straturi. Fiecare strat a fost
identificat si desfacut.
- ghici_strat(s) e folosit pentru ghicirea stratului
- desfa(s) e folosit pentru decodarea mesajului

Etapele de decodare

| Pas | Strat recunoscut | Intrare | Ieșire |
|-----|------------------|---------|--------|
| 1 | base64 | `NTM1OTU3NTk=` | `53595759` |
| 2 | hex | `53595759` | `SYWY` |
Al treilea pas dadea caractere non imprimabile, deci m-am oprit la pasul 2.

### 2, Cheie XOR
Fisierul probe/xor.bin e un texr criptat cu xor, cu o cheie de un octet. Am incercat
toate cheile de la 0 pana la 255. Pentru fiecare cheie am aplicat xor pe fiecare octet
si am verificat daca rezultatul e format de caractere imprimabile.

Prima cheie citibila:
> cheie:  128 ,  Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}

### Parola sparta
Fisierul probe/hashuri.txt contine hasuri. Lungimea lui arata ca e MD5. Am aplicat
atac prin dictionar. Pentru fiecare cuvant din probe/wordlist.txt am calculat hashul
si l-am comparat cu hashul tinta.

Primul cuvant care se potriveste:

> Gasit:  password
```

test_decode.md

```
import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
```

```
from Laborator6.arsenal.forensics.decode import *

TEST_DATA = {
    "%2Fetc": "url",
    "48656c6c6f": "hex",
    "U2FsdXQ=": "base64",
    "FLAG{x}": "necunoscut",
}

@pytest.mark.parametrize("sir, strat_asteptat", TEST_DATA.items())
def test_ghici_strat(sir, strat_asteptat):
    assert ghici_strat(sir) == strat_asteptat



def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"


def test_hex():
    strat, b = desfa("48656c6c6f")
    assert strat == "hex"
    assert b == b"Hello"


def test_url():
    strat, b = desfa("%2Fetc")
    assert strat == "url"
    assert b == b"/etc"

def test_strat_necunoscut_nu_se_desface():
    strat, b = desfa("FLAG{x}")
    assert strat == "necunoscut"
    assert b is None


def test_citibil_text_simplu():
    assert citibil(b"Salut") is True


def test_citibil_octeti_de_control():
    assert citibil(b"\x00\x01\x02") is False
```

Output:

```
============================= test session starts ==============================
collecting ... collected 10 items

Laborator6/teste/test_decode.py::test_ghici_strat[%2Fetc-url] PASSED     [ 10%]
Laborator6/teste/test_decode.py::test_ghici_strat[48656c6c6f-hex] PASSED [ 20%]
Laborator6/teste/test_decode.py::test_ghici_strat[U2FsdXQ=-base64] PASSED [ 30%]
```

```
Laborator6/teste/test_decode.py::test_ghici_strat[FLAG{x}-necunoscut] PASSED [ 40%]
Laborator6/teste/test_decode.py::test_base64 PASSED                   [ 50%]
Laborator6/teste/test_decode.py::test_hex PASSED                      [ 60%]
Laborator6/teste/test_decode.py::test_url PASSED                      [ 70%]
Laborator6/teste/test_decode.py::test_strat_necunoscut_nu_se_desface PASSED [ 80%]
Laborator6/teste/test_decode.py::test_citibil_text_simplu PASSED      [ 90%]
Laborator6/teste/test_decode.py::test_citibil_octeti_de_control PASSED   [100%]

============================== 10 passed in 0.02s ==============================
```

LUCRAREA B (extindere). Forensics pe fișiere și un mesaj ascuns

Obiectiv: citești ce spun fișierele despre ele (semnătură, text, metadate), extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Se lucrează pe fișierele din probe/.

B1. Magic bytes: tipul real, nu extensia (12 min)

Deschide un fișier în mod binar și citește-i primii octeți. Compară-i cu semnăturile cunoscute.

```
def tip_real(cale):
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    #       b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    #       b"PK\x03\x04" -> "zip/docx"
    return cap.hex()
print(tip_real("probe/foto.jpg"))
```

De observat. Modul "rb" nu e opțional pentru fișiere binare: fără el, octeții se pot altera.

B2. strings: textul citibil dintr-un binar (12 min)

Scoate secvențele de caractere imprimabile dintr-un fișier, cu un regex pe octeți.

```
import re
date = open("probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
#           print(m.decode())
# cauta un indiciu: o cale, un mesaj, un nume de fisier
```

B3. Metadate EXIF și GPS din poză (16 min)

Citește blocul EXIF din probe/foto.jpg și scoate coordonatele GPS. Convertește-le în grade zecimale.

```
from PIL import Image
from PIL.ExifTags import TAGS
exif = Image.open("probe/foto.jpg")._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
# TODO: ia blocul GPS (tag 34853) si tipareste
#       latitudinea si longitudinea
```

De observat. Coordonatele vin ca (grade, minute, secunde) plus emisfera. Le poți pune pe o hartă: de aici valoarea lor pentru OSINT, și motivul să ștergi EXIF înainte de a publica.

B4. File carving: scoți fișierul ascuns (14 min)

probe/ascuns.png are o arhivă ZIP lipită la coadă. Găsește-i semnătura și salveaz-o separat.

```
date = open("probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")
# TODO: daca i != -1:
#   open("gasit.zip","wb").write(date[i:])
#   print("arhiva extrasa de la octetul", i)
# apoi dezarhiveaza si vezi ce era inauntru
```

B5. Ascunzi un mesaj prin LSB (16 min)

Scrie un mesaj în ultimii biți ai pixelilor dintr-o poză gazdă. Rezultatul arată identic cu originalul.

```
from PIL import Image
img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)
# TODO: pentru fiecare bit, ia un canal si pune
#   canal = (canal & ~1) | int(bit)
# TODO: img.putdata(px_nou); img.save("stego.png")
```

De observat. Salvează în .png (fără pierdere); un .jpg recompresează imaginea și distruge biții ascunși.

B6. Dezvălui mesajul înapoi (14 min)

Citește ultimii biți ai pixelilor din stego.png, regrupează-i câte 8 și reconstituie mesajul.

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    # TODO: aduna ultimii biti (r&1, g&1, b&1)
    pass
# TODO: grupeaza cate 8 biti -> octeti -> text
#       opreste-te la primul octet 0 (final de mesaj)
```

De observat. Dezvăluirea e exact inversul ascunderii: aceiași pași, citiți invers. Când apare text citibil, ai găsit mesajul.

> **PENTRU VÂRFUL SĂLII (opțional).** Dacă termini devreme, în probe/ găsești challenge.bin: un fișier care combină tot ce ai făcut azi (un strat de encoding peste un XOR, cu un hash de verificat). Despică-l singur, fără schelet, și pune pașii în raport. Este genul de problemă pe care o vei întâlni pe pwn.college și pe challenge-urile de forensics.

Predai: scripturile de la B1-B6, fișierul stego.png, și în raport.md: tipul real al fișierelor, ce ai găsit în metadate (inclusiv GPS), ce era în arhiva ascunsă, și mesajul LSB dezvăluit.

forsenics.py

```
import re
from PIL import Image
from PIL.ExifTags import TAGS

print("B1:")
def tip_real(cale):
    cap = open(cale, "rb").read(8)

    if cap.startswith(b"%PDF"):
        return "pdf"
    if cap.startswith(b"\x89PNG"):
        return "png"
```

```
    if cap.startswith(b"\xff\xd8\xff"):
        return "jpeg"
    if cap.startswith(b"PK\x03\x04"):
        return "zip/docx"
    return cap.hex()

print(tip_real("../../probe/foto.jpg"))


print("\nB2:")

date = open("../../probe/ascuns.png", "rb").read()

for m in re.findall(rb"[ -~]{4,}", date):
    print(m.decode())

print("\nB3:")

def cord_zecimale(cord, emisfera):
    g, m, s = cord
    z = g + (m / 60.0) + (s / 3600.0)
    if (emisfera in ['W', 'S']):
        z = -z
    return z

exif = Image.open("../../probe/foto.jpg").getexif() or {}

gps_info = exif.get_ifd(34853) if hasattr(exif, "get_ifd") else {}

if gps_info:
    lat = gps_info.get(2)
    lat_ref = gps_info.get(1)
    lon = gps_info.get(4)
    lon_ref = gps_info.get(3)

if lat and lat_ref and lon and lon_ref:
    print(f'Lat: {cord_zecimale(lat, lat_ref)}')
    print(f'Long: {cord_zecimale(lon, lon_ref)}')

for id, val in exif.items():
    if (TAGS.get(id,id) != "GPSInfo"):
        print(TAGS.get(id, id), "=", val)


print("\nB4:")
date = open("../../probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")

if i != -1:
    open("gasit.zip", "wb").write(date[i:])
    print("arhiva extrasa de la octetul", i)



print("\nB5:")
```

```
img = Image.open("../../probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}" + b"\x00"
biti = "".join(f"{x:08b}" for x in mesaj)

px_nou = []
total_biti = len(biti)
i = 0


for p in px:
    if i>= total_biti:
        px_nou.append(p)
        continue

    canale = list(p)
    for c in range(3):
        if i < total_biti:
            canale[c] = (canale[c] & ~1) | int(biti[i])
            i +=1
    px_nou.append(tuple(canale))

img_noua = Image.new("RGB", img.size)
img_noua.putdata(px_nou)
img_noua.save("stego.png")
print("S-a salvat stego.png")

print("\nB6:")
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    biti += str(r & 1) + str(g & 1) + str(b & 1)

mesaj = bytearray()
for i in range(0, len(biti) - 7, 8):
    octet = int(biti[i:i+8], 2)

    if octet == 0:
        break
    mesaj.append(octet)

print(mesaj.decode("utf-8", errors="replace"))
```

Output:

```
B1:
jpeg

B2:
IHDR
IDATx
15hB
IEND
secret.txtFLAG{ascuns_prin_carving}
secret.txtPK

B3:
```

```
Lat: 47.016666666666666
Long: 28.8575
Model = iPhone 13
DateTime = 2026:09:14 10:30:00
Make = Apple

B4:
arhiva extrasa de la octetul 781

B5:
/home/xperiencez/Documents/GitHub/tehnici-de-
programare/Laborator6/arsenal/forensics/forsenics.py:70: DeprecationWarning:
Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use
get_flattened_data instead.
  px = list(img.getdata())
S-a salvat stego.png

B6:
/home/xperiencez/Documents/GitHub/tehnici-de-
programare/Laborator6/arsenal/forensics/forsenics.py:97: DeprecationWarning:
Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use
get_flattened_data instead.
  px = list(Image.open("stego.png").convert("RGB").getdata())
FLAG{ascuns_in_pixeli}
```

stego.png



raport.md

```
### 1. Tipul real al fisierrelor
Extensia unui fisier nu spune nimic garantat despre continutul lui. Scriptul citeste
primii 8 bytes si compara cu semnaturile cunoscute.

#### foto.jpg:
> Primii bytes al fisierului foto.jpg sunt FF D8 FF E0 00 10 4A 46. Asta este
semnatura JPEG. Tipul fisierului image.jpg corespunde extensiei.

### 2. Metadate (exif si gps)
```

```
Scriptul citeste metadatele EXIF din foto.jpg cu getexif(). Blocul GPS contine
coordonatele.

#### Metadate generale

> - Model = iPhone 13
> - Make = Apple
> - DateTime = 2026:09:14 10:30:00

#### Coordonate GPS
> - Lat: 47.016666666666666
> - Long: 28.8575


Concluzie - fotografia pastreaza coordonatele a locului unde a fost efectuata poza.
Daca ar fi publicata, oricine cu cunostintele date ar putea afla locul pozei.
Platformele social media in mod normal sterg aceste metadate.

### 3. Arhiva ascunsa in ascuns.png
Am cautat in fisierul ascuns.png cu semnatura ZIP PK\x03\x04. AM gasit-o la pozitia
byteului 781. Datele arhivei sunt de la aceasta pozitie pana la sfarsit.

Sirurile imprimabile din pasul 2 arata numele fisierului din arhiva si continutul sau:

> - secret.txtFLAG{ascuns_prin_carving}
> - secret.txtPK

Structura arhivei zip e alcatuit din fisierul secret.txt. La deschiderea lui vad
flagul urmator:

`FLAG{ascuns_prin_carving}`
```

CONCLUZII

Laboratorul a fost destul de interesant. Am vazut ca lucruri pe care le credem ascunse sau sigure pot fi de fapt usor de aflat, daca ai uneltele potrivite.

In prima parte, am scris un decoder care recunoaste si desface mai multe straturi de codare: urll, hex, base64. Am spart si o cheie xor de un singur octet, ceea ce a fost simplu, pentru ca am incercat pur si simplu toate cele 256 de valori. La final, am ghicit o parola md5 cu un atac prin dictionar, si a mers imediat, pentru ca parola era una simpla. Asta arata clar ca o parola slaba nu tine mult, si ca md5 e prea rapid ca sa protejeze parolele.

In a doua parte, am invatat ca extensia unui fisier nu spune mare lucru. Fotografia era un JPEG adevarat, iar in fisierul png era ascunsa o arhiva zip. Din metadatele fotografiei am aflat modelul telefonului, data si

coordonatele gps exacte de unde a fost facuta poza. Am scos si un mesaj ascuns cu steganografie LSB in subpixeli ce nu e vizibil cu ochii liberi.

Pe parcurs am mai comis si niste greseli. Unele funciti pareau ok dar afisau rezultate gresite, iar am prins asta doar uitandu-ma atent la output. Am inteles ca un rezultat care arata bine nu inseamna automat ca si codul e bun, si ca merita sa verific manual cateva exemple..

Pe scurt, ce-i important de stiut in continuare: trebuie de verificat tipul real al fisierelor, nu doar extensia; sterg metadatele inainte sa postez poze; si aleg parole care nu apar in dictionare.